

Chapter 31

ECDSA Digital Signature Peripheral (ECDSA_DS)

31.1 Introduction

In cryptography, the Elliptic Curve Digital Signature Algorithm (ECDSA) offers a variant of the Digital Signature Algorithm (DSA) which uses elliptic-curve cryptography.

ESP32-P4's ECDSA Digital Signature Peripheral (ECDSA_DS) provides a secure and efficient environment for computing ECDSA signatures. It offers fast computations while ensuring the confidentiality of the signing process to prevent information leakage. This makes it a valuable tool for applications that require high-speed cryptographic operations with strong security guarantees. By using ECDSA_DS, users can be confident that their data is protected without sacrificing performance.

31.2 Features

ESP32-P4's ECDSA_DS module supports:

- Digital signature verification
- Three different elliptic curves, namely, P-192, P-256, and P-384 defined in [FIPS 186-5](#)
- Multiple hash algorithms for message hash in the ECDSA_DS operation, including SHA-224, SHA-256, SHA-384, SHA-512, SHA-512-224, and SHA-512-256, defined in [FIPS PUB 180-4 Spec](#)
- State-dependent register access control in different operation states to ensure information security

31.3 ECDSA Basics

31.3.1 Domain Parameters

ECDSA uses parameters that define the elliptic curve over a finite field, as well as the generator point and the order of the base point. These parameters are usually referred to as domain parameters, and they are required for key generation, signature generation, and signature verification.

The domain parameters used in ECDSA consist of the following:

- The elliptic curve domain parameters, which include:
 - The prime modulus p , which specifies the size of the finite field over which the elliptic curve is defined.
 - The coefficients a and b , which specify the elliptic curve equation.
 - The base point G on the curve, which can be used to generate public keys.

- The order n of the base point, which is the number of points on the curve that can be generated by repeatedly adding G to itself.
- The parameters for the hash function, which include:
 - The hash algorithm used to generate a fixed-length hash value from the message being signed.
 - The length of the hash value, which determines the size of the signature.

Together, these parameters define the ECDSA domain.

31.3.2 Key Generation

The process of key generation in ECDSA is described below:

1. Select an elliptic curve domain by defining:
 - the prime modulus p
 - the coefficients a and b
 - the base point G and its order n
2. Generate a private key:
 - A private key d is a randomly chosen integer between 1 and $n-1$. It is essential to use a secure [Random Number Generator \(RNG\)](#) to ensure that the private key is truly random and cannot be predicted or reproduced.
 - The private key is used for signature generation. This key must be kept secret and secure.
3. Compute the public key:
 - The public key Q is calculated as $Q = dG$.
4. Export the public key:
 - The public key Q is typically represented as a pair of coordinates (Qx, Qy) that can be shared with others.

31.3.3 Signature Generation

The ECDSA signature generation process is described below:

1. Select a message m to be signed.
2. Calculate the hash of the message e : e is equal to $\text{HASH}(m)$, where HASH is a cryptographic hash function, such as SHA-256.
3. Compute the digest of the message hash z : Let z be the L_n leftmost bits of e , where L_n is the bit length of the base point order n .
4. Select a random number k , which is chosen between 1 and $n-1$, where n is the order of the base point on the elliptic curve.
5. Compute the signature: The signature is calculated as follows:
 - (a) Compute the point $(x, y) = kG$

- (b) Calculate $r = x \bmod n$. If r is equal to zero, return to the previous step and select a new random value of k .
 - (c) Calculate $s = k^{-1} * (z + d * r) \bmod n$. If s is equal to zero, return to step 4 and select a new random value of k .
 - (d) The signature is the pair (r, s) .
6. Send the message m and signature (r, s) to the recipient.

31.3.4 Signature Verification

The recipient can then use the public key associated with the private key used for signature generation to verify the signature. The verification process involves checking that the signature was generated using the correct private key and that the signature is valid for the given message.

The ECDSA signature verification process is described below:

1. Obtain public key Q and receive the message m and signature (r, s) .
2. Calculate the hash of the message e : e is equal to $\text{HASH}(m)$, where HASH is a cryptographic hash function, such as SHA-256.
3. Compute the digest of the message z : Let z be the L_n leftmost bits of e , where L_n is the bit length of the base point order n .
4. Verify the signature: The signature is verified as follows:
 - (a) Verify that r and s are integers between 1 and $n-1$, where n is the order of the base point on the elliptic curve. If either r or s is outside of this range, the signature is invalid.
 - (b) Calculate $u_1 = z * s^{-1} \bmod n$ and $u_2 = r * s^{-1} \bmod n$.
 - (c) Calculate the point $(x_1, y_1) = u_1 * G + u_2 * Q$, where G is the base point on the elliptic curve, and Q is the public key associated with the private key used for signature generation.
 - (d) Verify that $r = x_1 \bmod n$. If r is not equal to $x_1 \bmod n$, the signature is invalid.
5. Accept or reject the signature: If the signature is valid, the recipient can be confident that the message was not tampered with and that it came from the expected sender, thus can accept the message as authentic. Otherwise, the recipient rejects the message as invalid.

31.4 Functional Description

This section describes the details of ESP32-P4's ECDSA_DS module.

31.4.1 ECDSA_DS Working Modes

The ECDSA_DS module integrated in the ESP32-P4 supports Signature Verification mode.

Users can select the elliptic curves used by the ECDSA_DS module by configuring [ECDSA_ECC_CURVE](#) according to Table [31.4-1](#).

Table 31.4-1. ECDSA_DS Elliptic Curve Selection

ECDSA_ECC_CURVE	Elliptic Curve
0	P-192
1	P-256
2	P-384

Users can select the SHA algorithms for message hash by configuring [ECDSA_SHA_MODE](#) according to Table 31.4-2.

Table 31.4-2. ECDSA_DS SHA Algorithm

ECDSA_SHA_MODE	SHA Algorithm
1	SHA-224
2	SHA-256
3	SHA-384
4	SHA-512
5	SHA-512-224
6	SHA-512-256
Others	Invalid

Additionally, users can check the working states of the ECDSA_DS module by inquiring the [ECDSA_STATE_REG](#) register and comparing the return value against the Table 31.4-3 below.

Table 31.4-3. ECDSA_DS Working State

ECDSA_STATE_REG	State	Description
0	IDLE	Idle or completed operation. Corresponding to IDLE Stage.
1	LOAD	Waiting for users to load information into ECDSA_DS. Corresponding to LOAD Stage.
2	Reserved	–
3	BUSY	In the middle of a hardware operation. Corresponding to PREP, PROC & POST Stage.

31.4.2 Data and Data Block

ESP32-P4's ECDSA_DS module can operate on data of 192, 256, 384, or 512 bits. For example, the data ($D[255 : 0]$) can be divided into 32-bit data blocks.

Take 256-bit long data as an example, $D[n][31 : 0]$ ($n = 0, 1, \dots, 7$). Data blocks with the smaller serial number correspond to the lower binary bits. To be specific:

$$D[255 : 0] = D[7][31 : 0], D[6][31 : 0], D[5][31 : 0], D[4][31 : 0], D[3][31 : 0], D[2][31 : 0], D[1][31 : 0], D[0][31 : 0]$$

31.4.2.1 Writing Data

Writing data means writing data to an ECDSA_DS memory block and using this data as the input to ECDSA_DS. To be specific, writing data to an ECDSA_DS memory block means writing $D[n][31 : 0]$ to the “starting address of this ECDSA_DS memory block + $4 \times n$ ”. For a 256-bit long data:

- write $D[0]$ to “starting address”
- write $D[1]$ to “starting address + 4”
- ...
- write $D[7]$ to “starting address + 28”

Note:

When storing data, write only the data block of the required length. Do not append a 0 to the most significant bit. For example, for 192-bit data, store only the 192 bits without appending 0.

31.4.2.2 Reading Data

Reading data means reading data from the starting address of an ECDSA_DS memory block and using this data as the output from ECDSA_DS. To be specific, reading data from an ECDSA_DS memory block means reading $D[n][31 : 0]$ from the “starting address of this ECDSA_DS memory block + $4 \times n$ ”. For a 256-bit long data:

- read $D[0]$ from “starting address”
- read $D[1]$ from “starting address + 4”
- ...
- read $D[7]$ from “starting address + 28”

Note:

When reading data, only the data block of the required length needs to be read. For example, to read 192-bit data, simply read the lower 192 bits (i.e., 6 data blocks).

31.4.2.3 Padding the Message

The SHA accelerator can only process message blocks of 512 bits. Thus, all the messages should be padded to a multiple of 512 bits before the hash operation.

Suppose that the length of the message M is L_M bits. Then M shall be padded as introduced below:

1. First, append the bit “1” to the end of the message;
2. Second, append L_A bits of zeros, where L_A is the smallest, non-negative solution to the equation $L_M + 1 + L_A \equiv 448 \pmod{512}$;
3. Last, append the 64-bit block of value equal to the number L_M expressed using a binary representation.

For more details, please refer to [FIPS PUB 180-4 Spec](#) > Section “Padding the Message”.

31.4.2.4 Parsing the Message

The message and its padding must be parsed into N 512-bit message blocks: $M^{(1)}, M^{(2)}, \dots, M^{(N)}$.

Note:

1. For more details about “parsing the message”, please refer to [FIPS PUB 180-4 Spec](#) > Section “Parsing the Message”.
2. For more information on “message block”, please refer to [FIPS PUB 180-4 Spec](#) > Section “Glossary of Terms and Acronyms”.

31.4.3 Security Features

To ensure the security of the operation process, the ECDSA_DS module implements a variety of security functions.

31.4.3.1 High Anti-Attack Performance

ESP32-P4's ECDSA_DS module leverages ECC's enhanced anti-attack performance (refer to Section [26.4.3 Enhancing Anti-Attack Performance](#)) every time it performs an operation. This means that each time a signature is generated and verified by the ECDSA_DS:

- The execution latency is constant for a given operation.
- Power consumption variations are minimized.

This provides ESP32-P4's ECDSA_DS module strong anti-attack performance.

31.4.3.2 State-Dependent Register Access Control

ESP32-P4's ECDSA_DS module has implemented a state-dependent register access control mechanism to prevent any possibility of key theft by tampering with the configuration or accessing the data during the operation.

By implementing the state-dependent register access control, the accesses for ECDSA_DS registers are designed to vary in different states. For example, [ECDSA_CONF_REG](#) is only available for reading and writing when ECDSA_DS is in the IDLE state. In this way, the configuration information is protected from reading or writing when ECDSA_DS is in other states, such as LOAD and BUSY. For details about all ECDSA_DS working states, please refer to Table [31.4-3](#).

For detailed information on the state-dependent access control of each ECDSA_DS register, please refer to Section [Register Summary](#).

31.4.3.3 Hardware Occupation

During the operation, the following hardware modules will be occupied by ESP32-P4's ECDSA_DS module:

- SHA Accelerator
- ECC Accelerator

Among them, the SHA accelerator will be released when [ECDSA_SHA_RELEASE_INT](#) is triggered, while the ECC accelerator will be occupied during the whole ECDSA_DS operation.

Note:

Hardware occupation is a mechanism to protect multiplexed modules and storage space. When a module is hardware occupied, the user will fail to:

- read or write data to the module's registers or memories.
- disable the module clock.
- reset the module.

After hardware occupation is finished, the occupied module will be automatically reset. In addition, when the user performs a software reset to the master module, all the occupied modules will be reset at the same time.

31.5 Programming Procedures

31.5.1 ECDSA_DS Process

The overall ECDSA_DS process consists of the following five stages.

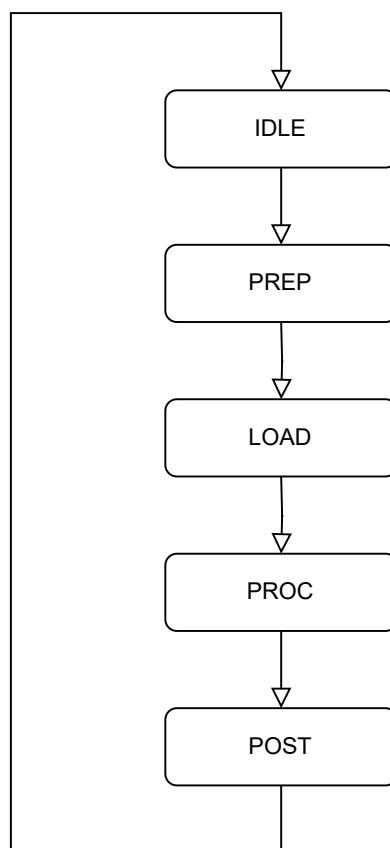


Figure 31.5-1. ECDSA_DS Process

The detailed programming procedures of each stage are described in the following sections.

31.5.1.1 IDLE Stage

In the IDLE stage:

1. Configure the static parameters including eFuse bits ECDSA_KEY, i.e., the value of private key d in ECDSA. To correctly configure the key value in eFuse, user need to write the key value in KEY n ($n = 0 \sim 5$), and set the corresponding EFUSE_KEY_PURPOSE_ n as ECDSA_KEY. Please refer to Chapter 8 [eFuse Controller \(EFUSE\)](#) for more detailed configuration steps.
2. Configure the configuration register, including the following fields:
 - (a) [ECDSA_ECC_CURVE](#): Choose the elliptic curve of the ECDSA_DS module.
 - (b) [ECDSA_SOFTWARE_SET_Z](#): Choose to use direct input z .
3. Configure the register field [ECDSA_START](#) to enter the PREP stage.

31.5.1.2 PREP Stage

In the PREP stage, [ECDSA_STATE_REG](#) is BUSY, and the ECDSA_DS performs preparation.

Wait till the PREP stage to end by polling [ECDSA_BUSY](#) until it is not BUSY. Then the ECDSA_DS will automatically enter the LOAD stage.

31.5.1.3 LOAD Stage

In the LOAD stage:

1. Provide input z into the ECDSA_DS using one of the following options:
 - Direct input z : write z to [ECDSA_Z_MEM](#).
 - ECDSA_DS SHA interface: generate z from the message. For details, please refer to Section 31.5.1.6.
2. Provide signature and public key required by signature verification:
 - (a) write signature (r, s) to [ECDSA_R_MEM](#) and [ECDSA_S_MEM](#).
 - (b) write public key (Qx, Qy) to [ECDSA_QX_MEM](#) and [ECDSA_QY_MEM](#).
3. Write 1 to [ECDSA_LOAD_DONE](#), indicating that the configuration is done. Then ECDSA_DS will automatically enter the PROC stage.

31.5.1.4 PROC Stage

In the PROC stage, [ECDSA_STATE_REG](#) is BUSY, and the ECDSA_DS performs ECDSA signature verification.

Wait till the PROC stage to end by polling [ECDSA_BUSY](#) until it is not BUSY. Then the ECDSA_DS will automatically enter the POST stage.

31.5.1.5 POST Stage

In the POST stage, [ECDSA_STATE_REG](#) is BUSY, and the ECDSA_DS performs wrap-up work of the operation.

Wait till the POST stage to end by polling [ECDSA_BUSY](#) until it is not BUSY. Then the ECDSA_DS will automatically return to the IDLE stage.

31.5.1.6 ECDSA_DS SHA Interface

ESP32-P4's ECDSA_DS module can automatically execute hash operation and generates z based on a direct input message.

For message hash, the ECDSA_DS module supports SHA algorithms SHA-224 (only valid when P-192 is selected as the elliptic curve) and SHA-256.

To use the ECDSA_DS SHA interface, complete the following steps:

1. Pad the message by following the steps described in Section [31.4.2.3](#).
2. Parse the message and its padding into message blocks. See details in Section [31.4.2.4](#).
3. Process the current message block.
 - Write the current message block into [ECDSA_MEM_M](#).
4. Start the ECDSA_DS SHA interface¹.
 - If this is the first time to execute this step, write 1 to [ECDSA_SHA_START](#) to start the ECDSA_DS SHA interface;
 - If this is not the first time to execute this step², write 1 to [ECDSA_SHA_CONTINUE](#) to continue the operation.
5. Check the progress of the current message block processing by polling.
 - Poll register [ECDSA_SHA_BUSY](#) until it's IDLE, indicating that the interface has completed the operation for the current message block.
6. Process the next message block:
 - If there is message block to process, go back to Step 3.
 - If there is no more message blocks, exit.

Note:

1. In this step, the software can also write the next message block to be processed, if any, in register [ECDSA_MEM_M](#), while the interface starts SHA operation, to save time.
2. You are resuming the ECDSA_DS SHA interface with the previously paused operation.

31.5.2 Clocks and Resets

ESP32-P4's ECDSA_DS module has one clock module and one reset module. It is activated by setting the HP_SYS_CLKRST_REG_CRYPT0_ECDSA_CLK_EN bit in the [HP_SYS_CLKRST_PERI_CLK_CTRL25_REG](#) register and clearing the HP_SYS_CLKRST_REG_RST_EN_ECDSA bit in the [HP_SYS_CLKRST_HP_RST_EN2_REG](#) register. For details on how to configure the ECDSA_DS clock and reset, please refer to Chapter [10 Reset and Clock](#).

31.5.3 Interrupts

ESP32-P4's ECDSA_DS module can generate the interrupt signal ECDSA_INTR and send it to [Interrupt Matrix](#).

The ECDSA_DS module has four interrupt sources that can generate the ECDSA_INTR interrupt signal as follows. Each interrupt source can be configured by a common set of registers that are described in Section [Interrupt Configuration Registers](#).

- ECDSA_PREP_DONE_INT: triggered on the completion of the PREP stage. This interrupt source is configured by the following registers:
 - [ECDSA_PREP_DONE_INT_RAW](#): stores the raw interrupt status of ECDSA_PREP_DONE_INT.
 - [ECDSA_PREP_DONE_INT_ST](#): indicates the status of the ECDSA_PREP_DONE_INT interrupt. This field is generated by enabling/disabling the [ECDSA_PREP_DONE_INT_RAW](#) field.
 - [ECDSA_PREP_DONE_INT_ENA](#): enables/disables the ECDSA_PREP_DONE_INT interrupt.
 - [ECDSA_PREP_DONE_INT_CLR](#): set this bit to clear the ECDSA_PREP_DONE_INT interrupt status. By setting this bit to 1, fields [ECDSA_PREP_DONE_INT_RAW](#) and [ECDSA_PREP_DONE_INT_ST](#) will be cleared.
- ECDSA_PROC_DONE_INT: triggered on the completion of the PROC stage. This interrupt source is configured by the following registers:
 - [ECDSA_PROC_DONE_INT_RAW](#): stores the raw interrupt status of ECDSA_PROC_DONE_INT.
 - [ECDSA_PROC_DONE_INT_ST](#): indicates the status of the ECDSA_PROC_DONE_INT interrupt. This field is generated by enabling/disabling the [ECDSA_PROC_DONE_INT_RAW](#) field.
 - [ECDSA_PROC_DONE_INT_ENA](#): enables/disables the ECDSA_PROC_DONE_INT interrupt.
 - [ECDSA_PROC_DONE_INT_CLR](#): set this bit to clear the ECDSA_PROC_DONE_INT interrupt status. By setting this bit to 1, fields [ECDSA_PROC_DONE_INT_RAW](#) and [ECDSA_PROC_DONE_INT_ST](#) will be cleared.
- ECDSA_POST_DONE_INT: triggered on the completion of the POST stage. This interrupt source is configured by the following registers:
 - [ECDSA_POST_DONE_INT_RAW](#): stores the raw interrupt status of ECDSA_POST_DONE_INT.
 - [ECDSA_POST_DONE_INT_ST](#): indicates the status of the ECDSA_POST_DONE_INT interrupt. This field is generated by enabling/disabling the [ECDSA_POST_DONE_INT_RAW](#) field.
 - [ECDSA_POST_DONE_INT_ENA](#): enables/disables the ECDSA_POST_DONE_INT interrupt.
 - [ECDSA_POST_DONE_INT_CLR](#): set this bit to clear the ECDSA_POST_DONE_INT interrupt status. By setting this bit to 1, fields [ECDSA_POST_DONE_INT_RAW](#) and [ECDSA_POST_DONE_INT_ST](#) will be cleared.
- ECDSA_SHA_RELEASE_INT: triggered when SHA is released. This interrupt source is configured by the following registers:
 - [ECDSA_SHA_RELEASE_INT_RAW](#): stores the raw interrupt status of ECDSA_SHA_RELEASE_INT.
 - [ECDSA_SHA_RELEASE_INT_ST](#): indicates the status of the ECDSA_SHA_RELEASE_INT interrupt. This field is generated by enabling/disabling the [ECDSA_SHA_RELEASE_INT_RAW](#) field via [ECDSA_SHA_RELEASE_INT_ENA](#).
 - [ECDSA_SHA_RELEASE_INT_ENA](#): enables/disables the ECDSA_SHA_RELEASE_INT interrupt.

- [ECDSA_SHA_RELEASE_INT_CLR](#): set this bit to clear the ECDSA_SHA_RELEASE_INT interrupt status. By setting this bit to 1, fields [ECDSA_SHA_RELEASE_INT_RAW](#) and [ECDSA_SHA_RELEASE_INT_ST](#) will be cleared.

Note:

For definitions of *interrupt*, *interrupt signal*, *interrupt source*, and their correlations, please refer to Chapter [12 Interrupt Matrix](#) > Section [12.2 Interrupt Terminology in ESP32-P4](#).

31.6 Memory Blocks

ECDSA_DS's memory blocks store input data and output data of the operation.

Table 31.6-1. ECDSA_DS Memory Blocks

Memory	Size (byte)	Starting Address [*]	Ending Address [*]	Access
ECDSA_MEM_M	64	0x280	0x2BF	R/W
ECDSA_MEM_R	32	0xA00	0xA1F	R/W
ECDSA_MEM_S	32	0xA20	0xA3F	R/W
ECDSA_MEM_Z	32	0xA40	0xA5F	R/W
ECDSA_MEM_Qx	32	0xA60	0xA7F	R/W
ECDSA_MEM_Qy	32	0xA80	0xA9F	R/W

^{*} Address offset related to the ECDSA_DS base address is provided in Table [7.3-2](#) in Chapter [7 System and Memory](#).

31.7 Register Summary

The addresses in this section are relative to the ECDSA_DS base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

To enhance security, ECDSA_DS registers have different read and write permissions in different operating states. The following is the abbreviation and corresponding relationship of each state:

- PI: IDLE state, corresponding to IDLE Stage.
- PL: LOAD state, corresponding to LOAD Stage.
- PB: BUSY state, corresponding to PREP, PROC and POST Stage.

Other abbreviations given in Column **Access** are explained in Section [Access Types for Registers](#).

Name	Description	Address	Access			
			PI	PL	PG	PB
Data Memory	See Table 31.6-1.					
Configuration Registers						
ECDSA_CONF_REG	ECDSA_DS configuration register	0x0004	R/W		N/A	
ECDSA_START_REG	ECDSA_DS start register	0x001C		WT		N
Clock and Reset Register						
ECDSA_CLK_REG	ECDSA_DS clock gate register	0x0008	R/W		N/A	
Interrupt Registers						
ECDSA_INT_RAW_REG	ECDSA_DS interrupt raw register	0x000C		RO/WTC/SS		
ECDSA_INT_ST_REG	ECDSA_DS interrupt status register	0x0010		RO		
ECDSA_INT_ENA_REG	ECDSA_DS interrupt enable register	0x0014		R/W		
ECDSA_INT_CLR_REG	ECDSA_DS interrupt clear register	0x0018		WT		
Status Registers						
ECDSA_STATE_REG	ECDSA_DS state register	0x0020		RO		
Result Register						
ECDSA_RESULT_REG	ECDSA_DS result register	0x0024		RO/SS		N
SHA Registers						
ECDSA_SHA_MODE_REG	ECDSA_DS controlling SHA register (Hash algorithm)	0x0200	N/A	R/W		N/A
ECDSA_SHA_START_REG	ECDSA_DS controlling SHA register (operation)	0x0210	N/A	WT		N/A
ECDSA_SHA_CONTINUE_REG	ECDSA_DS controlling SHA register (operation)	0x0214	N/A	WT		N/A
ECDSA_SHA_BUSY_REG	ECDSA_DS controlling SHA status register	0x0218	N/A	RO		N/A
Version Register						
ECDSA_DATE_REG	Version control register	0x00FC	R/W		N/A	

31.8 Registers

The addresses in this section are relative to the ECDSA_DS base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

For how to program reserved fields, please refer to Section *Programming Reserved Register Field*.

Register 31.1. ECDSA_CONF_REG (0x0004)

(reserved)																												ECDSA_SOFTWARE_SET_Z (reserved)				ECDSA_ECC_CURVE (reserved)																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																															
31																											6	5	4	3	2	1	0	Reset																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																													
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

ECDSA_ECC_CURVE Configures the elliptic curve used.

- 0: P-192
 - 1: P-256
 - 2: P-384
 - 3: Invalid
- (R/W)

ECDSA_SOFTWARE_SET_Z Configures how the parameter z is set.

- 0: Generated from SHA result
 - 1: Written by software
- (R/W)

Register 31.2. ECDSA_START_REG (0x001C)

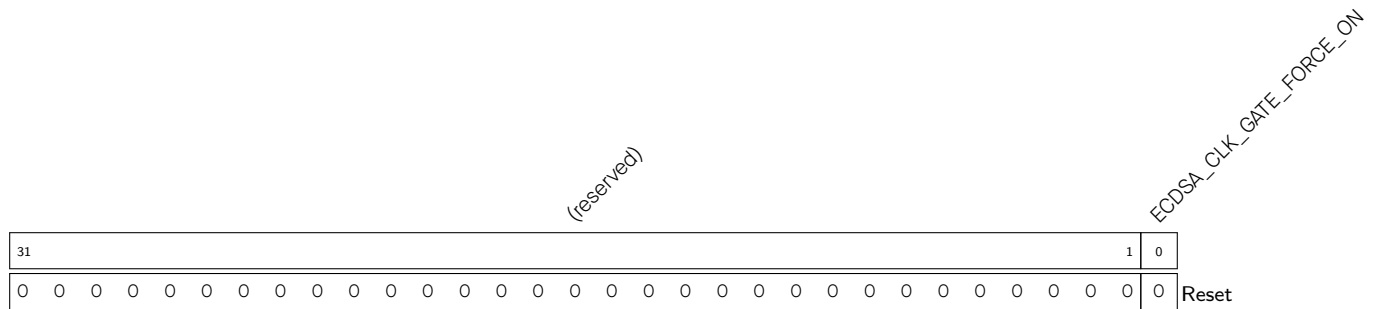
(reserved)																															ECDSA_LOAD_DONE		ECDSA_START																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																									
31																															2	1	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																									
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

ECDSA_START Configures whether to start the ECDSA_DS operation. This bit will be self-cleared after configuration.

- 0: No effect
 - 1: Start the ECDSA_DS operation
- (WT)

ECDSA_LOAD_DONE Write 1 to generate a signal indicating the ECDSA_DS's LOAD operation is done. This bit will be self-cleared after configuration. (WT)

Register 31.3. ECDSA_CLK_REG (0x0008)



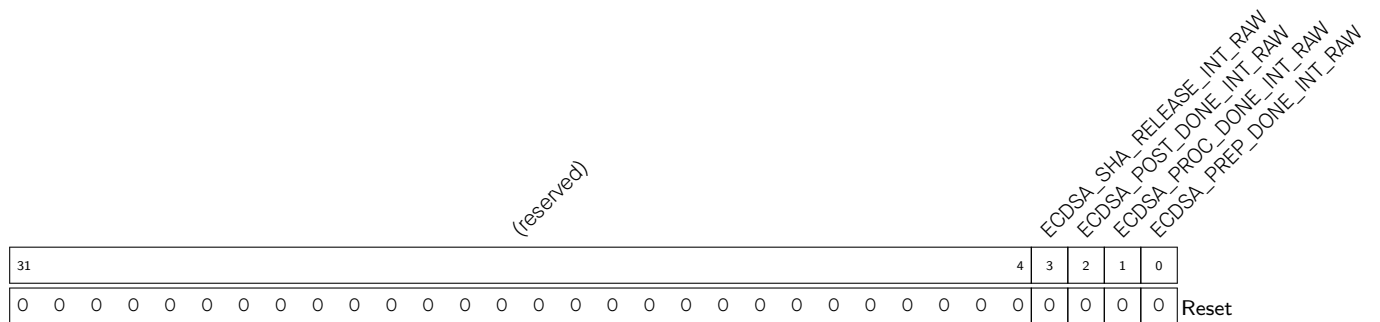
ECDSA_CLK_GATE_FORCE_ON Configures whether to force on ECC memory clock gate.

0: No effect

1: Force on

(R/W)

Register 31.4. ECDSA_INT_RAW_REG (0x000C)



ECDSA_PREP_DONE_INT_RAW The raw interrupt status of the [ECDSA_PREP_DONE_INT](#) interrupt.
(R/SS/WTC)

ECDSA_PROC_DONE_INT_RAW The raw interrupt status of the [ECDSA_PROC_DONE_INT](#) interrupt.
(R/SS/WTC)

ECDSA_POST_DONE_INT_RAW The raw interrupt status of the [ECDSA_POST_DONE_INT](#) interrupt.
(R/SS/WTC)

ECDSA_SHA_RELEASE_INT_RAW The raw interrupt status of the [ECDSA_SHA_RELEASE_INT](#) interrupt. (R/SS/WTC)

Register 31.5. ECDSA_INT_ST_REG (0x0010)

(reserved)																												ECDSA_SHA_RELEASE_INT_ST ECDSA_POST_DONE_INT_ST ECDSA_PROC_DONE_INT_ST ECDSA_PREP_DONE_INT_ST				
31																												4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset	

ECDSA_PREP_DONE_INT_ST The masked interrupt status of the [ECDSA_PREP_DONE_INT](#) interrupt. (RO)

ECDSA_PROC_DONE_INT_ST The masked interrupt status of the [ECDSA_PROC_DONE_INT](#) interrupt. (RO)

ECDSA_POST_DONE_INT_ST The masked interrupt status of the [ECDSA_POST_DONE_INT](#) interrupt. (RO)

ECDSA_SHA_RELEASE_INT_ST The masked interrupt status of the [ECDSA_SHA_RELEASE_INT](#) interrupt. (RO)

Register 31.6. ECDSA_INT_ENA_REG (0x0014)

(reserved)																										ECDSA_SHA_RELEASE_INT_ENA ECDSA_POST_DONE_INT_ENA ECDSA_PROC_DONE_INT_ENA ECDSA_PREP_DONE_INT_ENA			
31																					4	3	2	1	0				
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

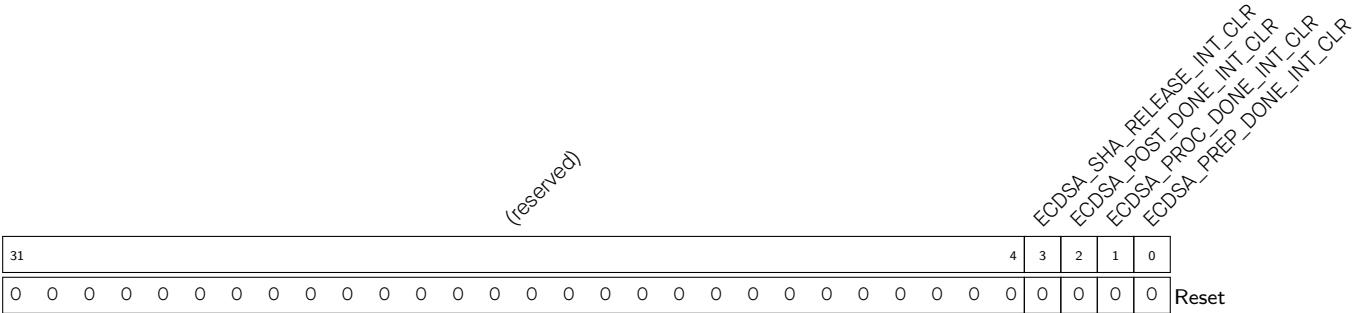
ECDSA_PREP_DONE_INT_ENA Write 1 to enable the [ECDSA_PREP_DONE_INT](#) interrupt. (R/W)

ECDSA_PROC_DONE_INT_ENA Write 1 to enable the [ECDSA_PROC_DONE_INT](#) interrupt. (R/W)

ECDSA_POST_DONE_INT_ENA Write 1 to enable the [ECDSA_POST_DONE_INT](#) interrupt. (R/W)

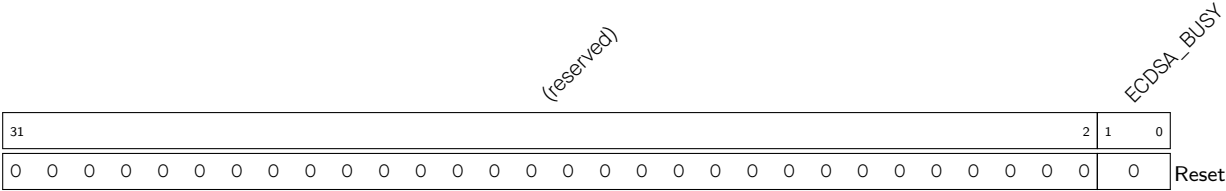
ECDSA_SHA_RELEASE_INT_ENA Write 1 to enable the [ECDSA_SHA_RELEASE_INT](#) interrupt. (R/W)

Register 31.7. ECDSA_INT_CLR_REG (0x0018)



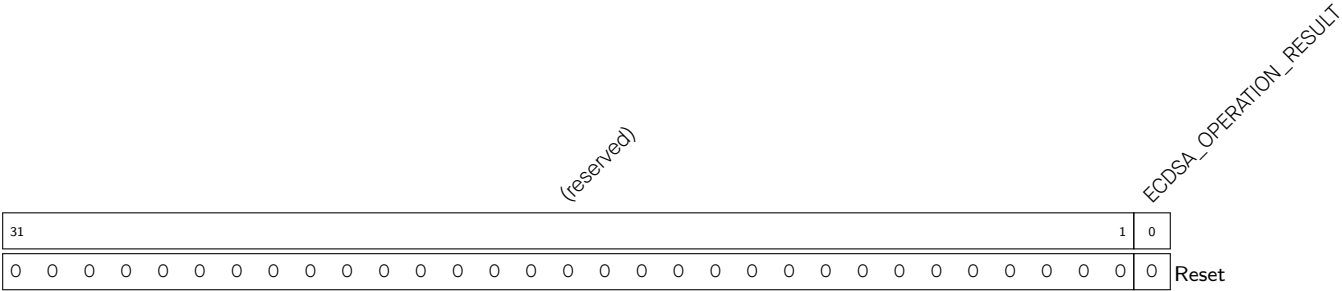
- ECDSA_PREP_DONE_INT_CLR Write 1 to clear the ECDSA_PREP_DONE_INT interrupt. (WT)
- ECDSA_PROC_DONE_INT_CLR Write 1 to clear the ECDSA_PROC_DONE_INT interrupt. (WT)
- ECDSA_POST_DONE_INT_CLR Write 1 to clear the ECDSA_POST_DONE_INT interrupt. (WT)
- ECDSA_SHA_RELEASE_INT_CLR Write 1 to clear the ECDSA_SHA_RELEASE_INT interrupt. (WT)

Register 31.8. ECDSA_STATE_REG (0x0020)



- ECDSA_BUSY Represents the working state of the ECDSA_DS module.
- 0: IDLE
- 1: LOAD
- 2: Reserved
- 3: BUSY
- (RO)

Register 31.9. ECDSA_RESULT_REG (0x0024)



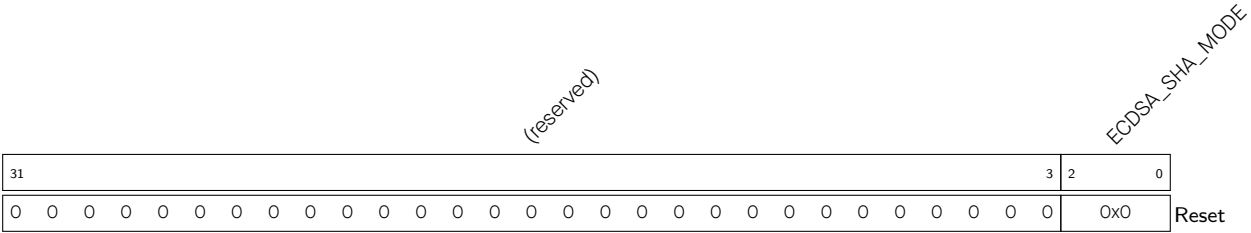
ECDSA_OPERATION_RESULT Indicates if the ECDSA_DS operation is successful.

- 0: Not successful
- 1: Successful

Only valid when the ECDSA_DS operation is done.

(RO/SS)

Register 31.10. ECDSA_SHA_MODE_REG (0x0200)



ECDSA_SHA_MODE Configures SHA algorithms for message hash.

- 1: SHA-224
- 2: SHA-256
- 3: SHA-384
- 4: SHA-512
- 5: SHA-512-224
- 6: SHA-512-256

Others: invalid

(R/W)

Register 31.11. ECDSA_SHA_START_REG (0x0210)

(reserved)																															ECDSA_SHA_START				
31																															1	0			
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

ECDSA_SHA_START Write 1 to start the first SHA operation in the ECDSA_DS process. This bit will be self-cleared after configuration. (WT)

Register 31.12. ECDSA_SHA_CONTINUE_REG (0x0214)

(reserved)																														ECDSA_SHA_CONTINUE	
31																														1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

ECDSA_SHA_CONTINUE Write 1 to start the latter SHA operation in the ECDSA_DS process. This bit will be self-cleared after configuration. (WT)

Register 31.13. ECDSA_SHA_BUSY_REG (0x0218)

(reserved)																														ECDSA_SHA_BUSY	
31																														1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

ECDSA_SHA_BUSY Represents the working state of the SHA accelerator in the ECDSA_DS process.

0: IDLE

1: BUSY

(RO)

Register 31.14. ECDSA_DATE_REG (0x00FC)

(reserved)				ECDSA_DATE																									
31		28		27																									0
0	0	0	0		0x2409180																								Reset

ECDSA_DATE The ECDSA_DS version control register. (R/W)